TJ_modul_7

Nama : Helven Marcelyno Aldy Dwy Anggara

Nim : 108092500012

Kelas : IT-08-02

FILE CODE

1. stack_character.h

stack_character.h | stack_character.cpp | main.cpp

```h
#ifndef STACK_CHARACTER_H_INCLUDED
#define STACK_CHARACTER_H_INCLUDED

#include <iostream>
#include <string>
using namespace std;

// Mendefinisikan konstanta batas maksimal stack sesuai modul
const int MAXSTACK = 10;
// Mendefinisikan tipe data elemen stack menjadi karakter (char)
typedef char infotype;

// Struktur dari stack[cite: 1]
struct stack {
    infotype info[MAXSTACK]; // Array untuk menyimpan elemen karakter[cite: 1]
    int top;                 // Indeks untuk menandai elemen paling atas stack[cite: 1
};

// ========================================
// Deklarasi Prosedur dan Fungsi Stack
// ========================================
void createStack(stack &S);
bool isEmpty(stack S);
bool isFull(stack S);
void push(stack &S, infotype xpush);
void pop(stack &S, infotype &xpop);
void printInfo(stack S);

// Deklarasi prosedur tambahan yang diminta di modul
void ascending(stack &S);
void descending(stack &S);
void stringToStack(stack &S, const string &str);
void reverseStack(stack &S);

#endif // STACK_CHARACTER_H_INCLUDED
```

2. stack_character.cpp

stack_character.h | stack_character.cpp | main.cpp

```cpp
#include "stack_character.h"

// Membuat stack baru dengan Top = 0 (Stack kosong)[cite: 2]
void createStack(stack &S) {
    S.top = 0; // Menginisialisasi nilai top dengan 0[cite: 2]
}

// Mengecek apakah stack kosong[cite: 2]
bool isEmpty(stack S) {
    // Jika Top(S) = 0, maka true, jika tidak maka false[cite: 3]
    return S.top == 0;
}

// Mengecek apakah stack sudah penuh[cite: 2]
bool isFull(stack S) {
    // Jika Top(S) mencapai MAXSTACK (10), maka true[cite: 3]
    return S.top == MAXSTACK;
}

// Memasukkan elemen baru ke dalam stack[cite: 2]
void push(stack &S, infotype xpush) {
    // Mengecek apakah stack belum penuh[cite: 3]
    if (isFull(S) == false) {
        S.top = S.top + 1;            // Menaikkan nilai top sebanyak 1[cite: 3]
        S.info[S.top - 1] = xpush;    // Memasukkan data ke array (dikurangi 1 karena C++ index mulai dari 0)
    } else {
        cout << "Stack penuh" << endl; // Pesan jika stack sudah mencapai MAXSTACK[cite: 3]
    }
}

// Mengeluarkan elemen teratas dari stack[cite: 2]
void pop(stack &S, infotype &xpop) {
    // Mengecek apakah stack tidak kosong[cite: 4]
    if (isEmpty(S) == false) {
        xpop = S.info[S.top - 1]; // Menyimpan nilai elemen paling atas ke xpop[cite: 4]
        S.top = S.top - 1;        // Menurunkan nilai top sebanyak 1[cite: 4]
    } else {
        cout << "Stack kosong" << endl; // Pesan jika stack kosong tidak bisa di-pop[cite: 4]
    }
}

// Menampilkan seluruh elemen stack dari atas ke bawah[cite: 2]
void printInfo(stack S) {
    if (isEmpty(S) == false) {
        // Melakukan perulangan dari elemen paling atas ke elemen paling bawah (1)[cite: 4]
        for (int i = S.top; i >= 1; i--) {
            cout << S.info[i - 1] << " "; // Print elemen (dikurangi 1 menyesuaikan index C++)
        }
        cout << endl;
    } else {
        cout << "Stack kosong" << endl; //[cite: 4]
    }
}

// ------------------------------------------------------
// IMPLEMENTASI FUNGSI TAMBAHAN
// ------------------------------------------------------

// Mengurutkan isi stack dari kecil ke besar (Ascending)[cite: 4]
void ascending(stack &S) {
    // Menggunakan algoritma Bubble Sort sederhana pada array Stack
    for (int i = 0; i < S.top - 1; i++) {
        for (int j = 0; j < S.top - i - 1; j++) {
            if (S.info[j] > S.info[j + 1]) {
                // Proses menukar (swap) data jika elemen sebelumnya lebih besar
                infotype temp = S.info[j];
                S.info[j] = S.info[j + 1];
                S.info[j + 1] = temp;
            }
        }
    }
}
```

```
}

// Mengurutkan isi stack dari besar ke kecil (Descending)[cite: 4]
void descending(stack &S) {
    // Menggunakan algoritma Bubble Sort sederhana
    for (int i = 0; i < S.top - 1; i++) {
        for (int j = 0; j < S.top - i - 1; j++) {
            if (S.info[j] < S.info[j + 1]) {
                // Proses menukar (swap) data jika elemen sebelumnya lebih kecil
                infotype temp = S.info[j];
                S.info[j] = S.info[j + 1];
                S.info[j + 1] = temp;
            }
        }
    }
}

// Menerima input string dan memasukkan tiap karakternya ke stack[cite: 4]
void stringToStack(stack &S, const string &str) {
    // Melakukan perulangan (looping) untuk setiap karakter dalam string
    for (int i = 0; i < str.length(); i++) {
        if (!isFull(S)) {
            push(S, str[i]); // Memasukkan karakter ke stack jika belum penuh
        } else {
            cout << "Stack penuh, sebagian string tidak masuk." << endl;
            break; // Hentikan loop jika stack penuh
        }
    }
}

// Membalikkan urutan isi stack dari atas ke bawah atau sebaliknya[cite: 5]
void reverseStack(stack &S) {
    int start = 0;              // Indeks paling bawah stack
    int end = S.top - 1;        // Indeks paling atas stack

    // Menukar elemen ujung ke ujung hingga saling bertemu di tengah
    while (start < end) {
        infotype temp = S.info[start];
        S.info[start] = S.info[end];
        S.info[end] = temp;
        start++;
        end--;
    }
}
```

3. main.cpp

stack_character.h | stack_character.cpp | main.cpp

```cpp
#include <iostream>
#include <string>
#include "stack_character.h"

using namespace std;

int main() {
    stack S1, S2, S3; //[cite: 6]
    infotype xpop;    //[cite: 6]

    // PENGUJIAN S1
    createStack(S1);  //[cite: 6]
    printInfo(S1);    //[cite: 6]
    push(S1, 'E');    //[cite: 6]
    printInfo(S1);    //[cite: 6]
    push(S1, 'T');    //[cite: 6]
    printInfo(S1);    //[cite: 6]
    push(S1, 'U');    //[cite: 6]
    printInfo(S1);    //[cite: 6]
    pop(S1, xpop);    //[cite: 6]
    printInfo(S1);    //[cite: 6]

    cout << endl << "Data diurutkan ascending:" << endl; //[cite: 6]
    ascending(S1); //[cite: 6]
    printInfo(S1); //[cite: 6]

    cout << endl << "Data diurutkan descending:" << endl; //[cite: 6]
    descending(S1); //[cite: 6]
    printInfo(S1);  //[cite: 6]

    // PENGUJIAN S2
    createStack(S2); //[cite: 6]
    string kalimat = "Hai kamu!!!"; //[cite: 6]
    cout << endl << "\'Hai kamu!!!\' di dalam stack:" << endl; //[cite: 6]
    stringToStack(S2, kalimat); //[cite: 6]
    printInfo(S2); //[cite: 6]

    cout << endl << "Data diurutkan ascending:" << endl; //[cite: 6]
    ascending(S2); //[cite: 6]
    printInfo(S2); //[cite: 6]

    cout << endl << "Data diurutkan descending:" << endl; //[cite: 6]
    descending(S2); //[cite: 6]
    printInfo(S2);  //[cite: 6]

    // PENGUJIAN S3
    createStack(S3); // Telah ditambahkan titik koma yang kurang di modul[cite: 6]
    cout << endl << "Membalik urutan elemen " << endl; //[cite: 6]
    string input; //[cite: 6]

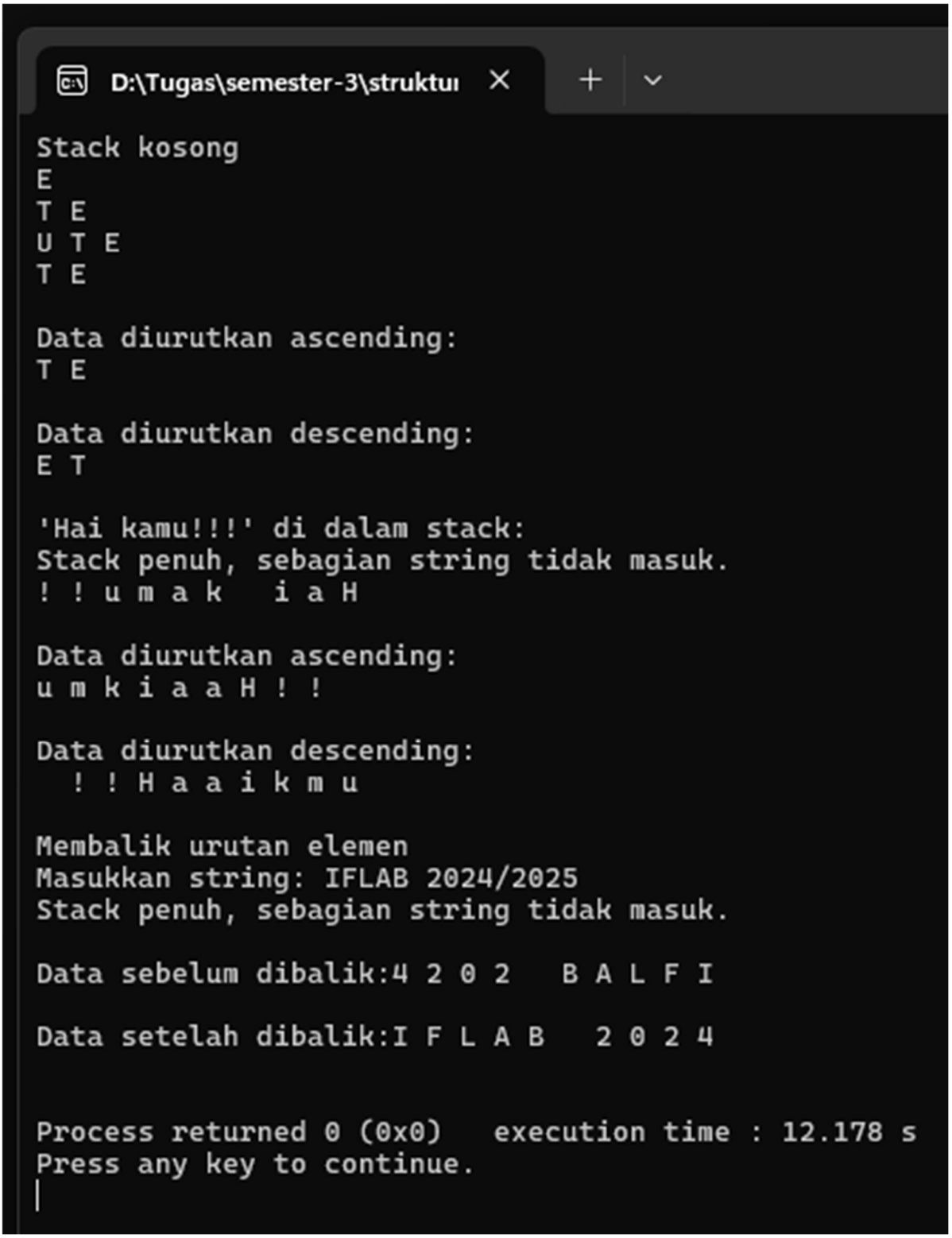
    cout << "Masukkan string: "; //
    // Inputkan "IFLAB 2024/2025"[cite: 7]
    getline(cin, input); // Menggunakan getline untuk membaca string dengan spasi[cite: 7]
    stringToStack(S3, input); //[cite: 7]

    cout << endl << "Data sebelum dibalik:"; //[cite: 7]
    printInfo(S3); //[cite: 7]
    cout << endl;  //[cite: 7]

    reverseStack(S3); // Telah ditambahkan titik koma yang kurang di modul[cite: 7]
    cout << "Data setelah dibalik:"; //[cite: 7]
    printInfo(S3); //[cite: 7]
    cout << endl;  //[cite: 7]

    return 0; //[cite: 7]
}
```

BUILD & RUN


D:\Tugas\semester-3\struktu
Stack kosong
E
T E
U T E
T E
Data diurutkan ascending:
T E
Data diurutkan descending:
E T
'Hai kamu!!!' di dalam stack:
Stack penuh, sebagian string tidak masuk.
! ! u m a k i a H
Data diurutkan ascending:
u m k i a a H ! !
Data diurutkan descending:
! ! H a a i k m u
Membalik urutan elemen
Masukkan string: IFLAB 2024/2025
Stack penuh, sebagian string tidak masuk.
Data sebelum dibalik:4 2 0 2 B A L F I
Data setelah dibalik:I F L A B 2 0 2 4
Process returned 0 (0x0) execution time : 12.178 s
Press any key to continue.